POS Project: Part Briefs

Oct 16, 2026

Shared rules

Five parts share one repo and one database; each part builds its own triggers, queries, endpoints and screens, and only the lead merges and deploys.

Stack. SQL Server Express on an Azure VM (T-SQL, `GO` separators), Node.js functions in `api/` on Vercel (no Express), React with Vite and React Router, `mssql`, bcrypt, jsonwebtoken and Jest.

Who owns which folder

Part	Owner	SQL	API	Screens
0 Skeleton	Elvis	`db/core/`	`lib/`, `api/auth/`	`src/app/`, `src/shared/`
1 Storefront	To assign	`db/storefront/`	`api/products/`, `api/cart/`, `api/reviews/`, `api/account/`, `api/jobs/`	`src/pages/storefront/`
2 Checkout and orders	To assign	`db/checkout/`	`api/checkout/`, `api/orders/`, `api/reports/sales.js`	`src/pages/checkout/`
3 Returns and rewards	To assign	`db/returns/`	`api/returns/`, `api/rewards/`, `api/reports/rewards.js`	`src/pages/returns/`
4 Back office	To assign	`db/backoffice/`	`api/manager/`, `api/reports/inventory.js`	`src/pages/backoffice/`

Report screens live in `src/pages/reports/`, one file per report, each added only by its owner.

Database rules

1. Triggers are `AFTER`, set-based and named `TR_<Table>_<Purpose>`. They start with `SET NOCOUNT ON; SET XACT_ABORT ON;` and fail with `THROW`. Every file begins with `USE pos_store;` then `GO`.
2. Never edit `CUSTOMER.Loyalty_points` or `Tier_id` directly. Insert a `LOYALTY_TRANSACTION` row and the trigger updates them.
3. Never recompute stock, points, tier or low-stock alerts in the app. The built triggers keep them in sync.
4. Every query is parameterized. Prices, tax and totals come from the database, never from the request.
5. A write that touches several tables runs in one database transaction, using the lead's helper.
6. Each trigger's tests go in `db/<part>/test.sql`. Every failing case runs in its own transaction and rolls back.

Rules decided on 2026-10-16

- Returns must be requested within 30 days of the sale; the database enforces it in `TR_PRODUCT_RETURN_ITEM_Guard`.
- Notifications are in-site only. `CUSTOMER.Notify_offers` and `Notify_cart` (default 1) let a customer turn each kind off.
- `PRODUCT.Is_active` and `DISCOUNT.Is_active` (default 1): deactivate instead of deleting. A discount applies only when `Is_active = 1` and today is within its dates.

- Discounts: per line, checkout takes the single largest saving among the product and category discounts. No stacking and no promo code box.
- A missing `TAX_RATE` for any item (category, ship-to state, sale date) fails checkout with a clear error. Never treat it as 0%.
- `Subtotal = SUM(Line_total)` (after discount, before tax) and `Total = Subtotal + Tax_amount`. There is no shipping fee.
- Loyalty points are earned and never spent. Only one reward program may be active at a time (`TR_REWARD_PROGRAM_NoOverlap`, already built).

What the lead provides

Names are proposed and become final at the end of Day 2.

- `lib/db.js`: `query(sql, params)` and `withTransaction(fn)`.
- `lib/auth.js`: `requireAuth(['manager'])(handler)`. The token carries the user id and role (customer, support or manager), and employee ids always come from the token, never the request body.
- `lib/http.js`: `sendJson` and `sendError`, which turns database errors (such as 547 for stock) into readable messages.
- `src/shared/`: layout, navigation per role, design tokens, buttons, cards, tables, spinner, empty state, error banner and an API client (`api.get`, `api.post`).
- A seed script with products, customers, employees and completed sales, by the end of Day 2.

Working agreement

- One branch per part (`part-1-storefront` and so on). Pull requests go to the lead, who merges to `main`. Vercel deploys from `main` only.
- Done means the SQL passes its tests, the endpoint has a Jest test that calls the handler directly, and the screen handles loading, empty and error states.
- If you fall behind, tell the lead early. Each part lists a must-have and a cut-first list so the lead can take over the must-haves.

Part 0: Skeleton (project lead)

The lead owns what the other four parts stand on (database, hosting, authentication, shared UI and the final merge) and must land the foundation by the end of Day 2.

Layer	Must-have	Cut first
Trigger	`TR_SALES_TRANSACTION_StatusFlow`: blocks any change away from `completed` or `voided`, so a finished sale can never be completed twice	None; it is small and protects trigger 1
Database	Already done on 2026-10-16: PRODUCT_RETURN fix, `Sent_at` rename, `Is_active` on PRODUCT and DISCOUNT, `Notify_offers` and `Notify_cart` on CUSTOMER, `TR_REWARD_PROGRAM_NoOverlap`. Still to do: seed script (ends with `UPDATE PRODUCT SET Stock_quantity = Stock_quantity;`, one reward program with tiers, a tax rate for every category in every test state); update `verify.sql` to 27 tables, 43 foreign keys and the trigger count; schema freeze at the end of Day 3	None; the schema changes are already applied
Hosting	Azure VM in East US with SQL Express (TCP/IP on, mixed-mode login, port 1433, `encrypt: true`); least-privilege app login, not `sa`; Vercel project, environment variables, security headers in `vercel.json`	Custom domain
Backend	`lib/db.js`, `lib/auth.js`, `lib/http.js`; customer and employee login, register, logout and `me`; bcrypt hashing; JWT in an httpOnly cookie; hand-written input validation helpers; login throttling	Throttling beyond a simple attempt counter
Frontend	Design tokens, layout and navigation per role, routing with protected routes, API client, shared components, login and register pages, home page, employee shell	Dark mode
Integration	Merge pull requests; run every `test.sql` together in `test_triggers.sql`; Jest suite for auth and roles; deployment; final `.bak` or `.sql`; updated conceptual schema document; README; demo script	Extra README sections

Others need from you by Day 2: the seed script (everyone) and the login and shared UI (everyone). The PRODUCT_RETURN fix and the `Sent_at` rename are already in the schema.

You need from others: their `triggers.sql` and `test.sql` files: Part 2's by Day 3 and the rest by Day 6, so the full chain can run through the app on Day 7.

Part 1: Storefront

Part 1 covers everything a customer sees before paying, plus their account and notifications, and it can start on Day 2 once login and seed data exist.

Owner: to assign. Branch: `part-1-storefront`.

Layer	Must-have	Cut first
Trigger	`TR_REVIEW_Guard`: when a review has a `Transaction_id`, it must belong to that customer, be `completed` and contain the product	The trigger itself; fall back to an app check
Queries	Catalog search and filter (name or SKU, category, condition, price range, sort) showing only active products (Is_active = 1) and the effective price after the larger of the active, in-date product and category discounts; product detail with average rating and reviews; cart add, update and remove through `VW_CART`; addresses (one default per customer) and phones	Phone management
Endpoints	`api/products/` (list, detail), `api/reviews/`, `api/cart/`, `api/account/` (profile, addresses, phones, notifications)	Posting reviews
Notification job	`api/jobs/notifications.js`, a protected endpoint that creates cart reminders (cart idle over 24 hours, no reminder in the last 3 days) and marks them sent. In-site only (decided). It skips customers whose Notify_cart (reminders) or Notify_offers (offers) is 0.	Offer notifications
Screens	Catalog with filters, product page with reviews, cart (links to `/checkout`), account (profile, addresses, phones, notification preferences), notifications	Phones section of the account page

Rules specific to this part

- The app sets `Added_at` in its own UPDATE whenever a cart quantity changes, so the idle-cart reminder sees real activity.
- The cart page only links to Part 2's `/checkout` route; it does not create orders.
- The lead decides how the job is triggered (Vercel Cron or a manual "run now" during the demo). Build it so either works.
- Catalog queries return only products with `Is_active = 1`; inactive products still show in old orders.
- The account page has one checkbox for `Notify_cart` and one for `Notify_offers`.

Others need from you: nothing blocks them, but Part 2 reads the cart through `VW_CART`, so agree its columns with Part 2 on Day 2.

Part 2: Checkout and orders

Part 2 turns a cart into a paid, completed order, and it is the critical path: nothing downstream (returns, loyalty, sales report) works until it does.

Owner: to assign (give this to the strongest teammate). Branch: `part-2-checkout`. Triggers and `sp_Checkout` due by the end of Day 3.

Layer	Must-have	Cut first
Triggers	`TR_TRANSACTION_ITEM_Guard`: items change only while the sale is `open`, and quantity must not exceed current stock. `TR_PAYMENT_Guard`: payments only on `open` sales, amount above 0, and payments never exceed the Total. `TR_SALES_TRANSACTION_PaymentCheck`: on `open` to `completed`, the sale needs at least one item and payments must equal the Total exactly	None
Queries	`sp_Checkout` (steps below); order history; order detail with items and payment	Order search
Report	Sales summary (completed sales only): revenue, orders and units by day and by category, plus top products	Category breakdown, month view
Endpoints	`api/checkout/` (POST), `api/orders/` (list, detail), `api/reports/sales.js` (managers only)	Order filters
Screens	Checkout (address, card, summary), confirmation, order history, order detail, sales report page	Entering a different ship-to address (use the default address)

What `sp_Checkout` does, in one database transaction

1. Insert the `SALES_TRANSACTION` as `open`, copying the ship-to address into its `Ship_*` columns.
2. Copy the cart rows into `TRANSACTION_ITEM` with the price snapshot and, per line, the single largest saving among the product and category discounts (both Is_active = 1 and in date; no stacking, no promo code) as the discount snapshot, then delete the cart rows.
3. Set Subtotal, Tax_amount and Total, with Subtotal = SUM(Line_total) (after discount, before tax), Total = Subtotal + Tax_amount, and tax looked up by each item's category, the ship-to state and the sale date in `TAX_RATE`.
4. Insert the `PAYMENT` rows. They must add up exactly to the Total.
5. Update the sale to `completed` with a unique tracking number and an estimated arrival date. A simple rule is enough (for example a generated code and a date 5 days out).

If any step fails (stock, payment mismatch), everything rolls back, including the stock change made by trigger 1.

Rules specific to this part

- Store only the payment method and `Card_last4`. Never store or log a full card number.
- If any item has no matching `TAX_RATE`, fail the whole checkout with a clear error. Never fall back to 0% tax.
- The request carries cart intent and a ship-to choice only; prices, tax and totals are always recomputed in the database.
- Tests (each in its own rolled-back transaction): edit items on a completed sale, overpay, complete an unpaid sale, and oversell (error 547).

Others need from you: completed sales for Part 3 to return against (the seed script covers this until Day 4), and an order detail page with a link to `/returns/new?order=<id>`.

Part 3: Returns and rewards

Part 3 handles money and points going back: customers request returns, support approves and refunds them, and the database restocks items and reverses loyalty points.

Owner: to assign. Branch: `part-3-returns`. The PRODUCT_RETURN fix is already applied, so nothing blocks you; use the seed script's completed sales until Part 2 is live.

Layer	Must-have	Cut first
Triggers	`TR_PRODUCT_RETURN_ITEM_Guard`: the sale must be `completed` and contain the product; the return must be within 30 days of the sale and still be `requested`; returned quantity across all non-denied returns of that sale and product must not exceed the purchased quantity; `Refund_amount` must not exceed the refund formula below. `TR_PRODUCT_RETURN_Refunded`: when a return becomes `refunded`, set `Refunded_at`, restock the returned items, and insert a negative `LOYALTY_TRANSACTION`	None
Queries	Create a return request; customer's returns; support queue of `requested` returns; approve, deny and mark refunded (the handler's `Employee_id` is saved at approval or denial); points balance, tier, next tier and history	Next-tier progress
Report	Rewards program: points issued and reversed, customers per tier, top earners	Top earners
Endpoints	`api/returns/` (customers create and list; support and managers approve, deny, refund), `api/rewards/`, `api/reports/rewards.js` (managers only)	Return reasons list
Screens	Request a return from an order (items, quantities, reason), my returns, rewards page (points, tier, history), support returns queue, rewards report page	Tier progress bar

Policies in force

- Refunds include proportional tax, and returned items go back to the same product at its original condition.
- Restocking adds the returned quantity to the product's `Stock_quantity`. That fires the low-stock trigger, which may close an open alert.
- Returns must be requested within 30 days of the sale. TR_PRODUCT_RETURN_ITEM_Guard enforces it by comparing the return's Requested_at with the sale's Transaction_date.
- A return moves `requested`, then `approved` or `denied`, then `refunded`. Items lock once the return leaves `requested`.

Refund formula

```
refund = round( (L + T * L / S) * q / n , 2 )
```

L is the line total of the product, T the sale's `Tax_amount`, S its `Subtotal`, q the returned quantity and n the purchased quantity. Subtotal is defined as the sum of the line totals (after discount, before tax), so this holds.

Points reversal. The negative loyalty row is `FLOOR(L * q / n * Points_per_dollar)`, because points were earned on the Subtotal and not on tax. Trigger 2 rejects a reversal larger than the current balance, so decide in the app how to handle that edge case.

Others need from you: nothing blocks them. You need completed sales from the seed script until Part 2 is live.

Part 4: Back office

Part 4 gives employees the tools to run the store: inventory, prices, discounts, low-stock alerts and supplier orders, with managers as the only role allowed to change inventory, prices and discounts.

Owner: to assign. Branch: `part-4-backoffice`.

Layer	Must-have	Cut first
Trigger	`TR_INVENTORY_ADJUSTMENT_Apply`: when an `INVENTORY_ADJUSTMENT` row is inserted, apply `Quantity_change` to the product's stock and reject a negative result	None
Queries	Product create, edit and activate or deactivate (Is_active); price edit that inserts a `PRICE_CHANGE` row in the same transaction; stock adjustment (insert the adjustment row only); discounts (with an Is_active pause switch) and sale events; low-stock alerts list; supplier orders with items; supplier and supplier-product lists	Sale events, supplier management
Report	Inventory and low stock: stock against threshold per product, open alerts with their age, supplier orders by status, inventory value at cost	Inventory value
Endpoints	`api/manager/` (products, prices, adjustments, discounts, sale-events, suppliers, supplier-orders, alerts), `api/reports/inventory.js`	Product delete
Screens	Employee dashboard (open alerts, pending supplier orders, returns waiting), inventory list with adjust, product edit with price, discounts, supplier orders, inventory report page	Sale event screens

The flow that must work end to end

1. A sale drops stock to the threshold and the database opens a `LOW_STOCK_ALERT`.
2. A manager sees the alert and creates a `SUPPLIER_ORDER` with its items. Nothing is ordered automatically.
3. The app links the alert to the order: `UPDATE LOW_STOCK_ALERT SET Order_id = @orderId WHERE Product_id = @productId AND Resolved_at IS NULL`.
4. The order moves `Pending`, then `Ordered`, then `Received` (`Cancelled` is allowed from `Pending` or `Ordered`). Items can change only while `Pending`.
5. Receiving fires the existing trigger that adds stock, which closes the alert once stock is above the threshold.

Rules specific to this part

- Manager role is required for inventory, price and discount writes; support can view. The employee id comes from the login token.
- Once `TR_INVENTORY_ADJUSTMENT_Apply` exists, the app never updates `Stock_quantity` directly.
- Deactivate products and discounts instead of deleting them: the catalog shows only active products, and a discount applies only when Is_active = 1 and today is within its dates. A hard delete of a product with history is blocked by foreign keys, so show a readable message.
- The app sends only valid supplier-order status changes; the database does not enforce the order.

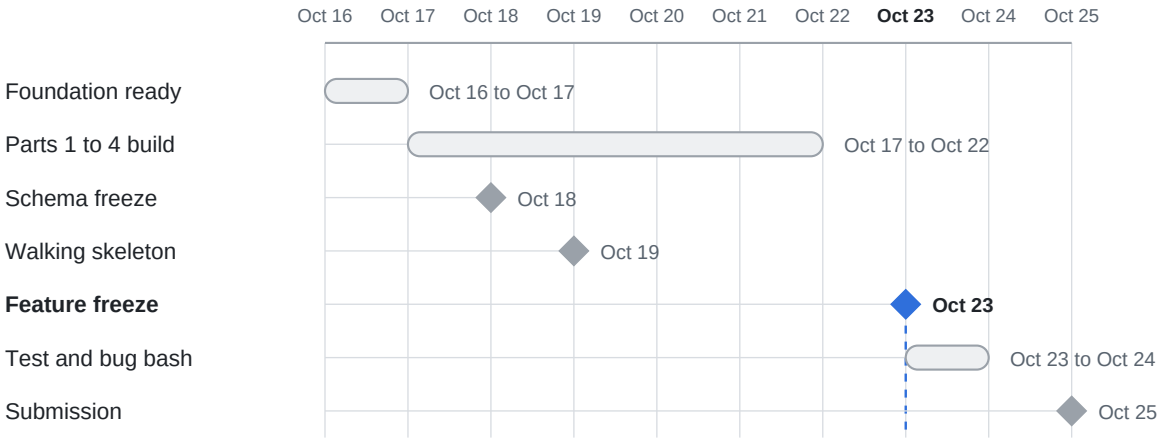
Others need from you: nothing blocks them. You need the employee shell and role checks from the lead, and alerts from the seed script until Part 2 is live.

Timeline and cut order

The feature freeze lands at the end of Day 8 (Fri Oct 23), which leaves Day 9 for a whole-team bug bash and Day 10 for submission.

Feature freeze on Oct 23 leaves two days for testing and submission

Project plan by day, Oct 16 to Oct 25, 2026



Two earlier checkpoints decide whether the rest is safe. On Day 1 a Vercel function must read from the Azure database. By the end of Day 4 the walking skeleton (log in, browse, add to cart, pay, see the order, all live on Vercel) must work. The lead freezes the schema at the end of Day 3, and no part changes tables after that without the lead's approval.

If a part falls behind

Cut in this order, and never cut the items in the last bullet.

1. `TR_REVIEW_Guard` (use an app check), offer notifications, sale event screens, phone management, posting reviews.
2. Supplier management screens, product delete, category and month breakdowns in reports, the tier progress bar, a custom ship-to address.
3. The notification job becomes a manual "run now" endpoint.

If a teammate misses their must-haves, the lead takes them over in this order: Part 2 (checkout), Part 4's alert-to-restock flow, Part 3's refund, then Part 1's catalog.

- **Never cut:** checkout with its three triggers, login and roles, the three reports (sales, rewards, inventory), the low-stock to supplier order to restock flow, and returns with refund, restock and points reversal.